

Computer Architecture: *Exceptions & Interrupts* *Part 1: Modes, Frames, C Code*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How do you observe the machine running?
- How do we define the machine?
- How does the machine control flow change (planned)?
- **How does the machine control flow change (unplanned)?**

...at some point: what is the *most basic possible* machine...

Control Flow So Far

```
14 | ldr r3, [fp, #-8]
    e3530003
18 | cmp r3, #3
    03a03001
1c | moveq r3, #1
    13a03000
20 | movne r3, #0
    e20330ff
24 | and r3, r3, #255 @ 0xff
    e3530000
28 | cmp r3, #0
    0a000001
2c | beq 38 <main+0x38>
    e59f0014
30 | ldr r0, [pc, #20] @ 4c <main+0x4c>
    ebfefefe
34 | bl 0 <printf>
    R_ARM_CALL printf
34 |
    e3a03000
```

1. Linear Flow
2. Branches (“**beq**”)
3. Subroutine calls (“**bl**”)

All of these are explicit & software-initiated.

Our Saviour: Exceptions



How do we make control flow change **without** a software-initiated request?

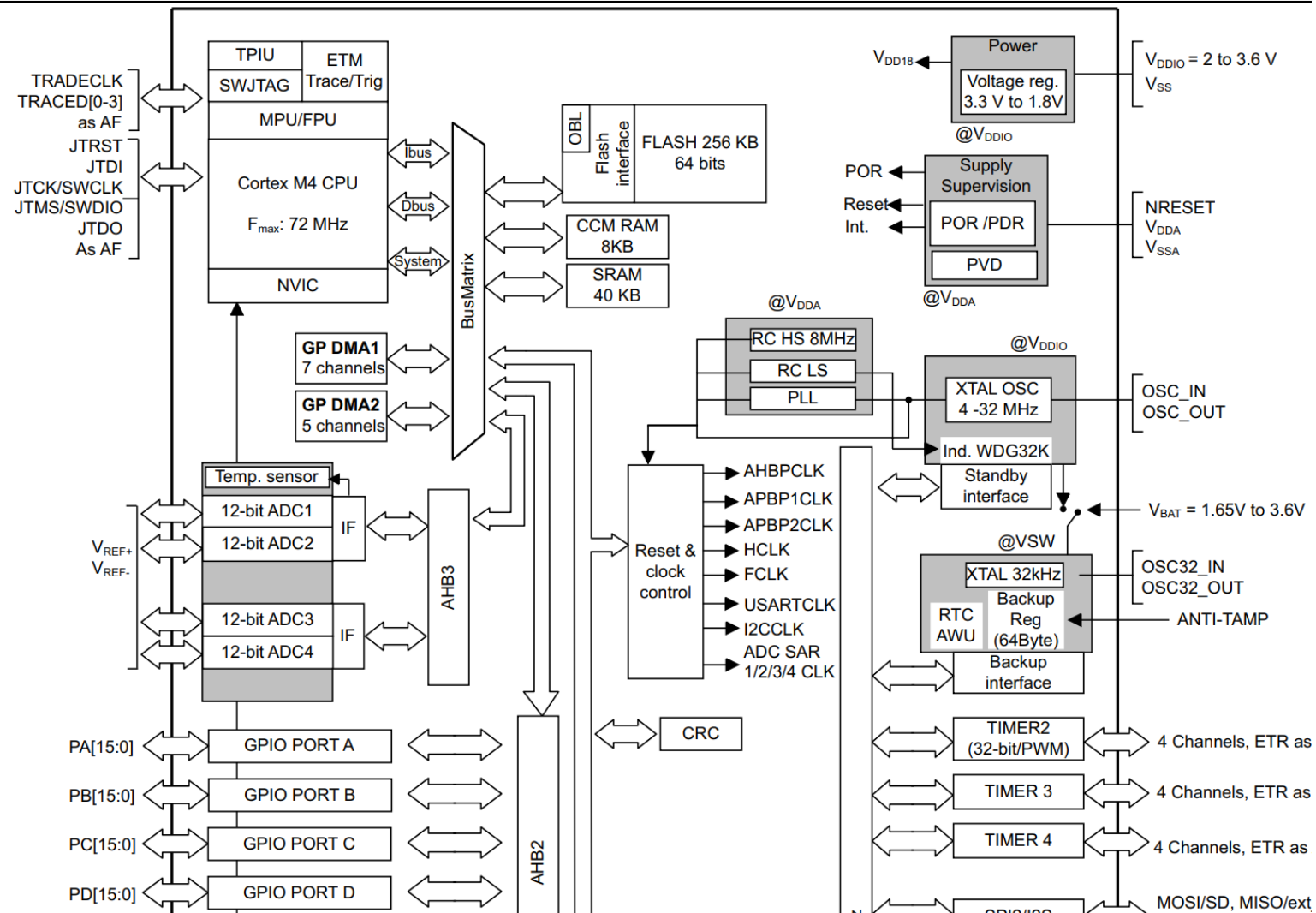
If software had to initiate every change we would need to **poll** for every possible scenario.

Interrupts vs. Exceptions

- Exceptions & Interrupts both result in hardware-level control flow change
- Interrupts are typically caused by some “external (to the CPU) event” (pin change, timer)
- Exceptions are typically caused by the CPU itself

What is the CPU?

Figure 1. STM32F303xB/STM32F303xC block diagram



Interrupt Vector Table

- Most (all?) processors have some form of a vector table which defines where control is transferred on an interrupt/exception.
- The level of hardware support is defined by the ISA.
 - Some processors at the hardware level are very simple – rely on a user to decode which interrupt triggered and call further routines.
 - Slower to pass control to a specific routine.
 - Some processors (e.g., Arm) are more complex and have very specific tables.
 - Faster to pass control to a specific routine.

Fast vs. Slow

Interrupt Address: In Table vs. Program Memory

- Some devices pass execution to the vector table location
- You need to have a “jmp” instruction there to pass control where you want.
- Some devices have an address in the vector table which tells the CPU where to pass control.

e.g., AVR: Code in vector table

11. Interrupts

This section describes the specifics of the interrupt handling as performed in Atmel® ATmega328P. For a general explanation of the AVR® interrupt handling, refer to [Section 6.7 "Reset and Interrupt Handling" on page 15](#).

- Each interrupt vector occupies two instruction words in Atmel ATmega328P.
- In Atmel ATmega328P, the reset vector is affected by the BOOTRST fuse, and the interrupt vector start address affected by the IVSEL bit in MCUCR.

11.1 Interrupt Vectors in ATmega328P

Table 11-1. Reset and Interrupt Vectors in ATmega328P

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x0002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow

Table 11-2 shows reset and interrupt vectors placement for the various combinations of BOOTRST and IVSEL settings. If the program never enables an interrupt source, the interrupt vectors are not used, and regular program code can be placed at these locations. This is also the case if the reset vector is in the application section while the interrupt vectors are in the boot section or vice versa.

Table 11-2. Reset and Interrupt Vectors Placement in ATmega328P⁽¹⁾

BOOTRST	IVSEL	Reset Address	Interrupt Vectors Start Address
1	0	0x0000	0x0002
1	1	0x0000	Boot reset address + 0x0002
0	0	Boot reset address	0x0002
0	1	Boot reset address	Boot reset address + 0x0002

Note: 1. For the BOOTRST fuse "1" means unprogrammed while "0" means programmed.

The most typical and general program setup for the reset and interrupt vector addresses in Atmel® ATmega328P is:

Address	Labels	Code	Comments
0x0000		jmp RESET	; Reset Handler
0x0002		jmp EXT_INT0	; IRQ0 Handler
0x0004		jmp EXT_INT1	; IRQ1 Handler
0x0006		jmp PCINT0	; PCINT0 Handler
0x0008		jmp PCINT1	; PCINT1 Handler
0x000A		jmp PCINT2	; PCINT2 Handler
0x000C		jmp WDT	; Watchdog Timer Handler
0x000E		jmp TIM2_COMPA	; Timer2 Compare A Handler
0x0010		jmp TIM2_COMPB	; Timer2 Compare B Handler
0x0012		jmp TIM2_OVF	; Timer2 Overflow Handler
0x0014		jmp TIM1_CAPT	; Timer1 Capture Handler
0x0016		jmp TIM1_COMPA	; Timer1 Compare A Handler
0x0018		jmp TIM1_COMPB	; Timer1 Compare B Handler
0x001A		jmp TIM1_OVF	; Timer1 Overflow Handler

https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf page 49+50

e.g., ARM: Addresses in vector table

the vector table for STM32F303xB/C/D/E and STM32F398xE devices.

Table 82. STM32F303xB/C/D/E, STM32F358xC and STM32F398xE vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	Fixed	Reset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	Fixed	HardFault	All classes of fault	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 0028

Address Contents

08000000	2000A000
08000004	08000B7B
08000008	0800077B
0800000C	0800026D
08000010	08000883
08000014	080009B3
08000018	080009B5
0800001C	00000000
08000020	00000000
08000024	00000000
08000028	00000000
0800002C	0800024D
08000030	080009B7
08000034	00000000

Default Interrupt Handler?

- Very common that systems have a *default handler*. May be used as one for everything or one per interrupt.
- May be an endless loop (will STOP execution) or a silent return.

```
300
301  /******
302  *
303  *   Dummy handler to be used for reserved interrupt vectors
304  *   and weak implementation of interrupts.
305  *
306  */
307      .section .init.Dummy_Handler, "ax"
308      .thumb_func
309      .weak Dummy_Handler
310      .balign 2
311 Dummy_Handler:
312     1: b 1b    // Endless loop
313
314
```

Normal Interrupt Types

- **Non-Maskable Interrupt (NMI)**
 - This interrupt cannot be “turned off” (masked)
- **Maskable Interrupt** (aka everything else)
- Interrupts are normally *asynchronous* as they are external to the instruction stream (generated outside CPU)

Exception Types

- Exceptions are normally *synchronous* as generated by CPU
- Faults such as:
 - Invalid instructions
 - Invalid memory access
 - Divide by zero
- Software Interrupt (SWI) used for control flow

e.g., STM32F3 NVIC Vectors

the vector table for STM32F303xB/C/D/E, STM32F358xC and STM32F398xE devices.

Table 82. STM32F303xB/C/D/E, STM32F358xC and STM32F398xE vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	Fixed	Reset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	Fixed	HardFault	All classes of fault	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 0028

e.g., STM32F3 NVIC Vectors

Table 82. STM32F303xB/C/D/E, STM32F358xC and STM32F398xE vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
-	3	Settable	SVCall	System service call via SWI instruction	0x0000 002C
-	5	Settable	PendSV	Pendable request for system service	0x0000 0038
-	6	Settable	SysTick	System tick timer	0x0000 003C
0	7	Settable	WWDG	Window watchdog interrupt	0x0000 0040
1	8	Settable	PVD	PVD through EXTI Line16 detection interrupt	0x0000 0044
2	9	Settable	TAMPER_STAMP	Tamper and TimeStamp interrupts through EXTI Line19	0x0000 0048
3	10	Settable	RTC_WKUP	RTC wakeup timer interrupt through EXTI Line20	0x0000 004C
4	11	Settable	FLASH	Flash global interrupt	0x0000 0050
5	12	Settable	RCC	RCC global interrupt	0x0000 0054
6	13	Settable	EXTI0	EXTI Line0 interrupt	0x0000 0058

Exceptions Defined in the ISA

- Exceptions / Interrupts are required to deal with:
 - I/O handling
 - Error handling
 - Real-time requirements
- Work-arounds for a non-exception processor:
 - Registers store state of things to deal with
 - Need to poll these registers – either spend lots of time polling or have long response times.

Exceptions & Jumps

- Exceptions are a hardware-initiated subroutine call with some level of automatic “context saving”

Interrupt Handlers in C Code

- Most frameworks will have some way of setting up interrupts.
- This is often different per architecture, compiler, and even framework.
- See documentation – I'll be showing you the most basic way here.
- Often framework uses a `__weak__` attribute to make life easier

e.g., Demo of Exceptions in C Code

Things to cover (notes to self)

- Interrupt table example
- C `__weak__` attribute
- Calling function and overriding the `__weak__`

```

154  *
155  *****
156  */
157
158  /*****
159  *
160  * Setup of the vector table and weak definition of interrupt handlers
161  *
162  */
163      .section .vectors, "ax"
164      .code 16
165      .balign 512
166      .global _vectors
167  _vectors:
168      //
169      // Internal exceptions and interrupts
170      //
171      VECTOR __stack_end__
172      VECTOR Reset_Handler
173      EXC_HANDLER NMI_Handler
174      VECTOR HardFault_Handler
175  #ifdef __ARM_ARCH_6M__
176      ISR_RESERVED
177      ISR_RESERVED
178      ISR_RESERVED
179  #else
180      EXC_HANDLER MemManage_Handler
181      EXC_HANDLER BusFault_Handler
182      EXC_HANDLER UsageFault_Handler
183  #endif
184      ISR_RESERVED
185      ISR_RESERVED
186      ISR_RESERVED
187      ISR_RESERVED
188      EXC_HANDLER SVC_Handler
189  #ifdef __ARM_ARCH_6M__
190      ISR_RESERVED
191  #else
192      EXC_HANDLER DebugMon_Handler
193  #endif
194      ISR_RESERVED
195      EXC_HANDLER PendSV_Handler
196      EXC_HANDLER SysTick_Handler
197      //
198      // External interrupts
199      //
200  #ifndef __NO_EXTERNAL_INTERRUPTS
201      ISR_HANDLER WWDG_IRQHandler
202      ISR_HANDLER PVD_IRQHandler
203      ISR_HANDLER TAMP_STAMP_IRQHandler
204      ISR_HANDLER RTC_WKUP_IRQHandler
205      ISR_HANDLER FLASH_IRQHandler
206      ISR_HANDLER RCC_IRQHandler

```

Vector File

(Notes to Self)

```
1  #include <stdio.h>
2
3  // In a library or startup file (e.g., startup_device.c)
4  void Default_Handler(void) {
5      printf("OH NO: Default Handler Tripped\n");
6      while(1);
7  }
8
9  //void SomeFunction(void) __attribute__((weak, alias("Default_Handler")));
10
11
12  /* Example of using __attribute__((weak)) */
13  __attribute__((weak)) void SomeFunction(void) {
14      printf("OH NO: Unimplemented Code Tripped\n");
15      while(1);
16  }
```

Add a file to project (stuff.c)

```
92
93
94  void SomeFunction(void);
95
96  void SomeFunction(void){
97      printf("Oh Yeah");
98      while(1);
99  }
100
101  int main(void) {
102      int i;
103
104      SomeFunction();
105
106  }
```

In main.c

(Notes to Self)

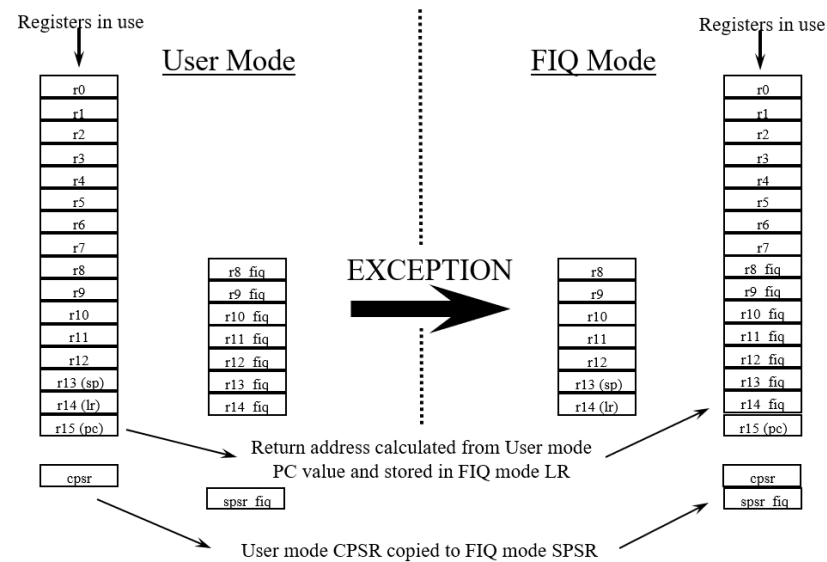
```
122  
123  
124  asm volatile("svc #0");  
125
```

```
asm volatile("svc #0");
```

Interrupt Context

From lecture on ISA:

e.g., User to Fast Interrupt (FIQ)



2026-02-09

ECED3403 Computer Architecture. Colin O'Flynn.

23

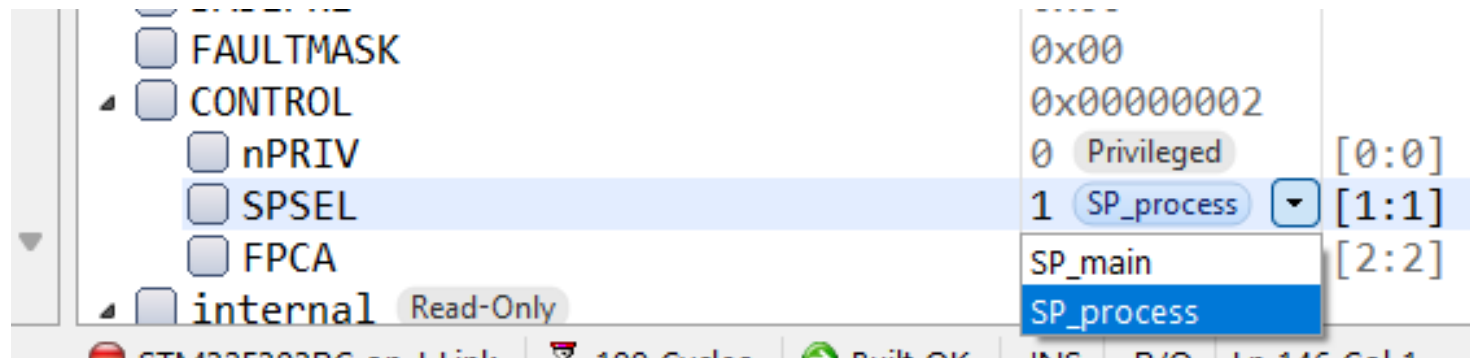
Cortex-M has a much simpler interrupt handler *without* register banks.

Interrupt Context

- Cortex-M operate in two modes:
 - Thread Mode:
 - Can run in both unprivileged & privileged mode
 - Handler Mode:
 - Always in *privileged mode* – full access to registers, memory etc

Stack Pointer Switching

- Cortex-M has two “stack pointers” – this just banks out r13 (SP).
- Compiling normal code always uses `SP_main` (even in interrupt)



Stack Pointer: Main vs. Process

- A RTOS will normally use Main Stack Pointer (MSP) for the Kernel, and Process Stack Pointer (PSP) for the processes.
- The interrupt handler uses the **current stack pointer** to save state, then switches to the **main stack pointer** then returns to the **previous stack pointer mode**.
- *This can be confusing if using both modes!*

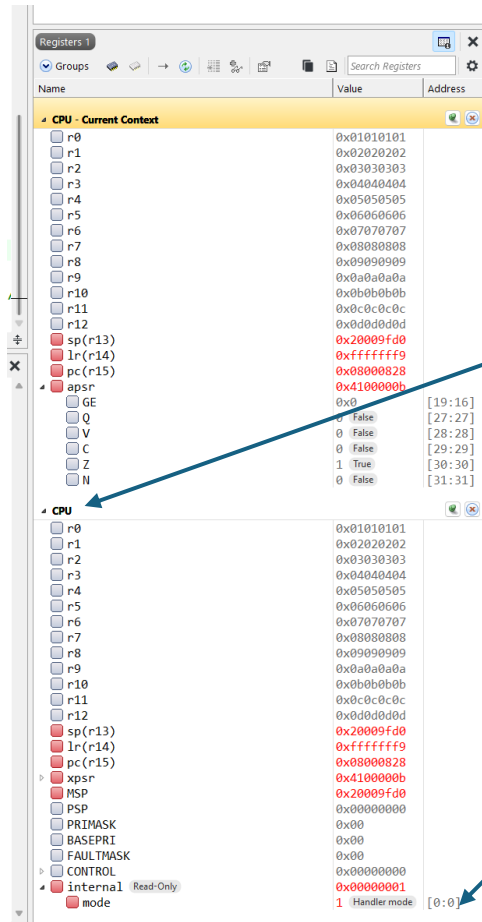
e.g., Demo of Exception Context (Notes to Self)

- Overwriting r15
- Overwrite r13 after changing main vs. process mode
- Show stack in both modes when calling interrupt
- Show registers being pushed onto stack

e.g., Demo of Exception Context

```
asm volatile("ldr r0, =0x01010101");
asm volatile("ldr r1, =0x02020202");
asm volatile("ldr r2, =0x03030303");
asm volatile("ldr r3, =0x04040404");
asm volatile("ldr r4, =0x05050505");
asm volatile("ldr r5, =0x06060606");
asm volatile("ldr r6, =0x07070707");
asm volatile("ldr r7, =0x08080808");
asm volatile("ldr r8, =0x09090909");
asm volatile("ldr r9, =0x0A0A0A0A");
asm volatile("ldr r10, =0x0B0B0B0B");
asm volatile("ldr r11, =0x0C0C0C0C");
asm volatile("ldr r12, =0x0D0D0D0D");
//Do not change r13 - that is stack pointer!
asm volatile("ldr r14, =0x0E0E0E0E");
```

e.g., Demo of Exception Context



Expand this

See mode

e.g., Demo of Exception Context

```
asm volatile("ldr r0, =0x01010101");
asm volatile("ldr r1, =0x02020202");
asm volatile("ldr r2, =0x03030303");
asm volatile("ldr r3, =0x04040404");
asm volatile("ldr r4, =0x05050505");
asm volatile("ldr r5, =0x06060606");
asm volatile("ldr r6, =0x07070707");
asm volatile("ldr r7, =0x08080808");
asm volatile("ldr r8, =0x09090909");
asm volatile("ldr r9, =0x0A0A0A0A");
asm volatile("ldr r10, =0x0B0B0B0B");
asm volatile("ldr r11, =0x0C0C0C0C");
asm volatile("ldr r12, =0x0D0D0D0D");
//Do not change r13 - that is stack pointer!
asm volatile("ldr r14, =0x0E0E0E0E");
```

e.g., Demo of Exception Context

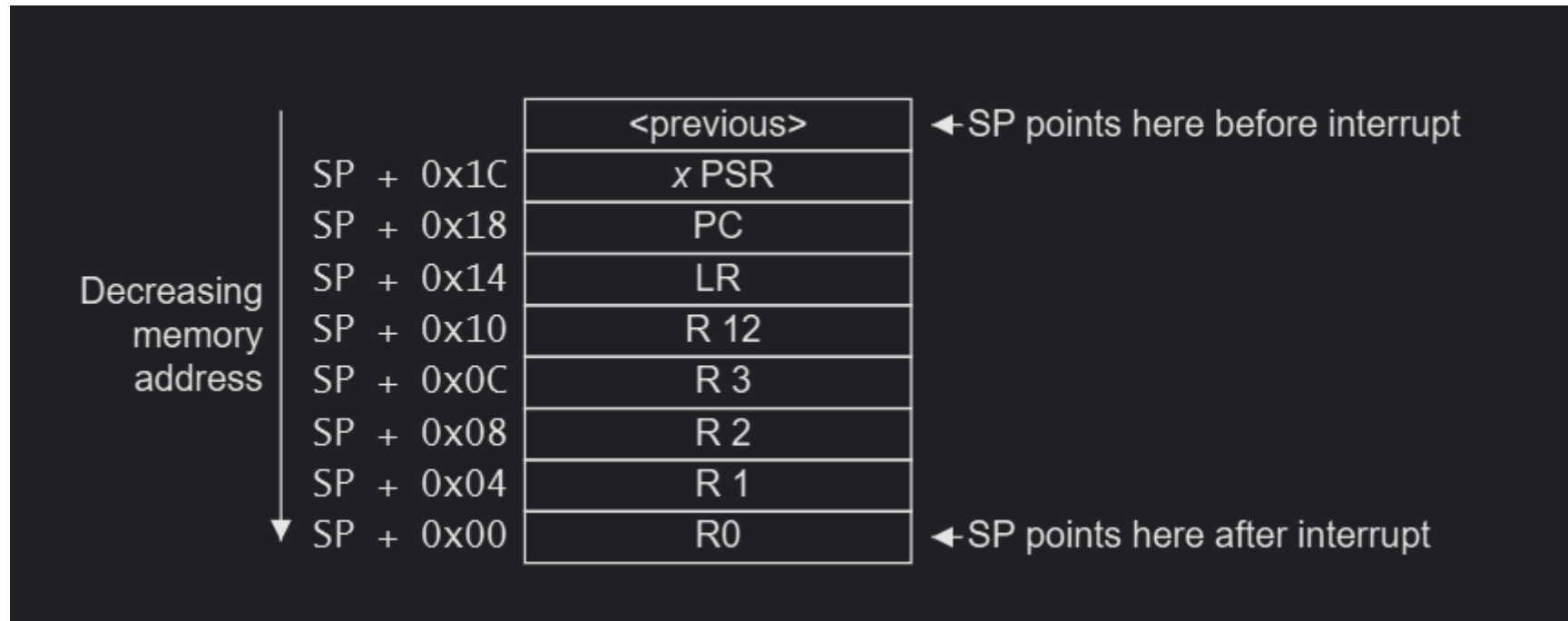
```
asm volatile("ldr r0, =0x01010101");  
asm volatile("ldr r1, =0x02020202");  
asm volatile("ldr r2, =0x03030303");  
asm volatile("ldr r3, =0x04040404");  
asm volatile("ldr r4, =0x05050505");  
asm volatile("ldr r5, =0x06060606");  
asm volatile("ldr r6, =0x07070707");  
asm volatile("ldr r7, =0x08080808");  
asm volatile("ldr r8, =0x09090909");  
asm volatile("ldr r9, =0x0A0A0A0A");  
asm volatile("ldr r10, =0x0B0B0B0B");  
asm volatile("ldr r11, =0x0C0C0C0C");  
asm volatile("ldr r12, =0x0D0D0D0D");  
//asm volatile("ldr r13, =0x0E0E0E0E");  
asm volatile("ldr r14, =0x0F0F0F0F");
```

```
asm volatile("nop");  
asm volatile("nop");  
asm volatile("nop");  
asm volatile("nop");
```

```
asm volatile("ldr r13, =0x20001000");
```

```
asm volatile("nop");  
asm volatile("nop");  
asm volatile("nop");  
asm volatile("nop");
```


Exception Stack Register



What is that LR Value?

Table 2.17. Exception return behavior	
EXC_RETURN	Description
0xFFFFFFFF1	Return to Handler mode. Exception return gets state from the main stack. Execution uses MSP after return.
0xFFFFFFFF9	Return to Thread mode. Exception Return get state from the main stack. Execution uses MSP after return.
0xFFFFFFFDD	Return to Thread mode. Exception return gets state from the process stack. Execution uses PSP after return.
All other values	Reserved.

<https://developer.arm.com/documentation/dui0552/a/the-cortex-m3-processor/exception-model/exception-entry-and-return>

e.g., Demo of Loading PC Directly

```
164  }  
165  
166  void SVC_Handler(void) {  
167      asm volatile("ldr r15, =0xFFFFFFFF9");  
168  }  
169
```

Demo Summaries (Hopefully)

- What registers went onto stack:
 - R0, R1, R2, R3, R12, R14 (LR), Return Address, Status Register (PSR)
- What is value in LR?
 - 0xffffffff9 → Special value indicating return mode
- Only two stack pointers in Cortex-M

e.g., AVR Interrupt Differences

Datasheet Interrupt Section – **READ FOR A NEW DEVICE**

When an interrupt occurs, the global interrupt enable I-bit is cleared and all interrupts are disabled. The user software can write logic one to the I-bit to enable nested interrupts. All enabled interrupts can then interrupt the current interrupt routine. The I-bit is automatically set when a return from interrupt instruction – RETI – is executed.

There are basically two types of interrupts. The first type is triggered by an event that sets the interrupt flag. For these interrupts, the program counter is vectored to the actual interrupt vector in order to execute the interrupt handling routine, and hardware clears the corresponding interrupt flag. Interrupt flags can also be cleared by writing a logic one to the flag bit position(s) to be cleared. If an interrupt condition occurs while the corresponding interrupt enable bit is cleared, the interrupt flag will be set and remembered until the interrupt is enabled, or the flag is cleared by software. Similarly, if one or more interrupt conditions occur while the global interrupt enable bit is cleared, the corresponding interrupt flag(s) will be set and remembered until the global interrupt enable bit is set, and will then be executed by order of priority.

The second type of interrupts will trigger as long as the interrupt condition is present. These interrupts do not necessarily have interrupt flags. If the interrupt condition disappears before the interrupt is enabled, the interrupt will not be triggered.

When the AVR exits from an interrupt, it will always return to the main program and execute one more instruction before any pending interrupt is served.

Note that the status register is not automatically stored when entering an interrupt routine, nor restored when returning from an interrupt routine. This must be handled by software.

e.g., AVR Interrupt Differences

- **AVR** does NOT save anything for you:
 - **Not even the status register**
- Pushes the return address to the stack only, everything else is up to you.
- As you use different devices be aware that there is few “universal truths”.

Summary

- Interrupts are external to CPU, normally asynchronous.
- Exceptions are generated by the CPU, normally synchronous.
- Interrupts/Exceptions use a vector table to tell the CPU where execution should pass.
- Normally some level of hardware support to save/restore context
 - May be minimal (AVR saving only return address) or more complete (Cortex-M saving multiple registers)